Web Recommender Systems - Final Submission

Carolina Courtin
University of Copenhagen
nlt840@alumni.ku.dk

1 Week 6 : Familiarize Yourself with the Datasets

For this first part, the dataset was mainly explored to understand what kind of data it contains, how the data are distributed, and what their main properties are.

We use the Video Games Amazon Review Data 2023 (5-core subset) dataset. This dataset contains user interactions in the form of reviews and ratings for video game products. For the purpose of this project, we rely on a curated and reduced version of the original dataset, provided through the course platform. This subset has been specifically designed to ensure computational feasibility while preserving the key characteristics of the full dataset. The dataset comprises a total of 37,500 interactions, involving 1,389 unique users and 932 unique items. Each interaction corresponds to a user reviewing a specific video game. To facilitate model development and evaluation, the dataset is provided with predefined splits :

- a training set (train_video_games.parquet) containing 80% of the interactions
- a test set (test_video_games.parquet) containing the remaining 20%

Both files are stored in Parquet format and are processed using the data manipulation tool Pandas. In addition, a metadata file (meta_video_games.parquet) is provided and will be used in later stages of the project. This file contains a description of the items attributes.

The first step consisted in cleaning the dataset by removing missing values and duplicate interactions. Table 1 shows how many data instances were affected by this cleaning process.

For the next check, a verification about all users appearing in the test set being also present in the training set was conducted. This is important to ensure that recommendations are only generated for users seen during training. No test interactions had to be removed, as all test users were already included in the training data.

Description	Number of instances
Initial dataset size	26,580
Missing user or item ID	0
Duplicate user–item interactions	0
Final dataset size after cleaning	26,580

Table 1: Description of the cleaned dataset

A first analysis of the training data was conducted by plotting the number of ratings for each item. This showed a clear long-tail distribution for both items and users. A small number of items receive a lot of ratings, while most items are rated only a few times. Similarly, most users interact with only a few items, while a small group of users is very active. This imbalance can affect recommendation performance. Popular items are easier to recommend because they have more interactions, while less popular items are harder to recommend due to sparse data. However, niche items can also be valuable, as they may better match specific user interests. Recommendation algorithms may also be biased toward popular items, since they learn that more popular usually means better predictions. In contrast, less popular items often suffer from data sparsity, which can result in weaker recommendations for niche content. Figure 1 shows this specific distribution.

Additionnaly, the relatively high standard deviation compared to the mean indicates that the dis-

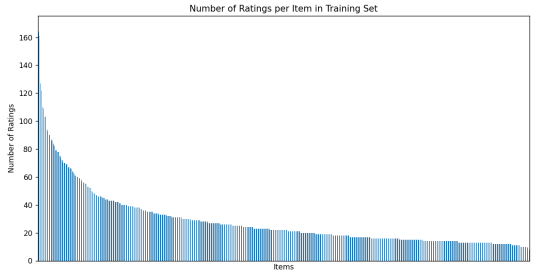


Figure 1: Long-tail distribution of items and their ratings

tribution of interactions is highly dispersed. This means that user activity and item popularity vary significantly across the dataset, with some users and items deviating strongly from the average behavior. Such heterogeneity can influence model performance, as recommendation algorithms may perform better on well-represented users and popular items while struggling with less frequent ones. Furthermore, the dataset is characterized by a relatively high minimum number of interactions for both users and items (12 and 8 respectively). In this regard, the dataset does not include true cold-start scenarios, making the recommendation task easier and potentially leading to overly optimistic evaluation results. Table 2 presents the results of these statistics.

Statistic	Items	Users
Total count	932	1,389
Mean	28.52	19.14
Standard deviation	21.09	10.20
Minimum	8	12
25th percentile	15	13
Median (50%)	22	16
75th percentile	33	21
Maximum	167	114

Table 2: Descriptive statistics of interactions per item and per user in the training set

The amount of times where each item was rated highly (items with a rating over 3) in the training set was computed. Table 3 shows the top-5 most highly rated items, along with the number of high ratings and the average rating for each item.

Item ID	High Ratings	Average Rating
B0086VPUHI	154	4.69
B00BN5T30E	150	4.63
B07YBXFDYN	146	4.71
B00BGA9WK2	136	4.72
B007CM0K86	123	4.68

Table 3: Top 5 most highly rated items in the training set

2 Week 7 : Collaborative Filtering Recommender System

The TopPop Recommender System is a non personalized recommendation approach that always suggests the same top-k items to every user, based on their popularity in the training set. Popularity is measured here by the number of high ratings. In the implementation, we select the k items with the highest counts of high ratings and recommend them to all users. This simple method serves as a baseline for evaluating other recommender systems.

An implementation of a neighborhood-based model using K-Nearest Neighbors was conducted. In this approach, predictions for a user are made based on the ratings of similar users (user-based) or similar items (item-based). We tested two similarity measures (cosine and Pearson) and different numbers of neighbors (k from 1 to 20). The best combination was selected using 5-fold cross-validation, with RMSE and MAE as evaluation metrics.

For evaluation, two common error metrics were used : Mean Absolute Error (MAE) and Root Mean Squared Error (RMSE). MAE calculates the average absolute difference between predicted and actual ratings, while RMSE gives the square root of the average squared differences. RMSE penalizes larger errors more than MAE, which makes it more sensitive to big mistakes in predictions. This is why RMSE is preferred here, as it better highlights models that make large errors. Table 4 shows the results of the best performing KNN models according to the RSME evaluation method.

Model	Parameters	RMSE	MAE
User-Based	k=19, cosine	0.9816	0.6816
Item-Based	k=19, cosine	0.9874	0.6846
User-Based	k=7, Pearson	1.0496	0.7028
Item-Based	k=10, Pearson	1.0821	0.7441

Table 4: Best KNN models and hyperparameters selected using RMSE

For comparison, we also implemented a latent factor model using Singular Value Decomposition (SVD). In this model, each user and item is represented by a set of latent factors learned from the observed ratings. We tuned the number of latent factors (n_factors) and the number of training epochs (n_epochs), using 5-fold cross-validation. The search space was defined as n_factors = 10, 20, 50 and n_epochs = 20, 30, 50. These parameters control the complexity of the model and the training process : increasing them can improve performance but may also lead to overfitting. The optimal configuration based on RMSE was obtained for $n_factors = 10$ and $n_epochs = 30$, achieving an RMSE of 0.9566 and a MAE of 0.6803. Results for the SVD model selection are shown in table 5.

Model	Parameters	RMSE	MAE
SVD	n_factors=10 n_epochs=30	0.9566	0.6803

Table 5: Best SVD model selected using RMSE

In our study, models with the best predictions in terms of rating accuracy were selected based on RMSE. However, if the main goal were to optimize the ranking of recommended items, we could instead choose a model based on higher Precision@k or Recall@k values. This distinction highlights that a model good at predicting ratings is not necessarily the one that produces the most relevant top recommendations for users.

Once the best hyperparameters for each model were computed, the models were then trained on the entire training set. We then tested them by generating the top recommendations for all users. Tables 6, 7 and 8 display examples of recommendations results using each models, specifically by generating the recommendations for the specific user "AHZDB5H32C3FZTMUPCYWWVRJINPQ".

Item ID	Predicted Rating
B08N7RDR45	4.51
B07CD6F5PX	4.45
B019EWMXWC	4.44
B079C7QGLQ	4.28
B001EYUNE4	4.27

Table 6: Top-5 recommendations for using KNN User-Based

Item ID	Predicted Rating
B07L6MJ6LD	5.00
B07F1HTGV1	5.00
B0C3KYVDWT	5.00
B0053BCO00	5.00
B01AC3ZD06	5.00

Table 7: Top-5 recommendations using KNN Item-Based

Item ID	Predicted Rating
B0002A6CQ4	4.29
B0029LJIFG	4.24
B01GY35GIM	4.21
B001EYUNPI	4.18
B0191J0RXA	4.18

Table 8: Top-5 recommendations using SVD

3 Week 8 : Evaluation of Recommender Systems

Now that we have the three models, we can see how well they actually perform on the test data. In this step, we compare the recommendations made by each model with the real ratings in the test set and calculate the average error between them. Table 11 shows these results for each model.

Model	RMSE on test set
UserCF KNN	0.9465
ItemCF KNN	0.9378
SVD	0.9295

Table 9: RMSE of each models on the test set

The biggest limitation of RMSE is that it is a very strict measure, because it penalizes large errors much more than small ones (since the errors are squared). This means that even a few badly predicted ratings can make the overall score worse than it really is. RMSE also tends to focus more on popular items, because they have more ratings, and it can ignore less-rated, niche items in the long

tail. So, while RMSE is useful to see how accurate a model is overall, it can sometimes be too harsh and not reflect the usefulness of recommendations for all items.

Now that we have computed the RMSE, we will look at additional evaluation metrics on the recommendations to better understand how the system performs and which aspects are strong or weak. The following metrics are used :

- Precision@k : The fraction of recommended items in the top-k list that are actually relevant to the user. Higher values mean more accurate recommendations.
- Hit Rate : Measures how often at least one of the top-k recommended items is relevant. It shows whether the system can recommend something useful to the user.
- MAP@k (Mean Average Precision at k) : Evaluates the quality of the ranking of the recommended items. It gives higher scores if relevant items appear earlier in the top-k list.
- MRR@k (Mean Reciprocal Rank at k) : Focuses on the first relevant item in the recommendation list. The sooner a relevant item appears, the higher the score.
- Coverage : The fraction of all items in the catalog that appear in any users top-k recommendations. It shows how diverse the recommendations are.

Table 10 and 11 shows the results for each of these metrics with k = 10. Looking at the results, TopPop has the highest Precision, Hit Rate, MAP, and MRR, which means it is very effective at recommending the most popular items that users are likely to enjoy. However, its coverage is very low, meaning it ignores almost all other items in the catalog. This is expected, as TopPop mainly focuses on the most popular items and does not consider the long tail of less popular items.

KNN models (both user-based and item-based) have the lowest performance in terms of Precision, Hit Rate, MAP, and MRR. This can be explained by the fact that similarity-based methods rely heavily on local neighborhoods, which can be sparse and noisy in real-world datasets. As a result, they struggle to rank the most relevant items at the top positions. However, KNN models achieve much higher coverage compared to SVD and TopPop. This is because they recommend items based on local similarities rather than global popularity or latent factors. Consequently, they are more likely to explore a wider range of items, including less popular ones. In contrast, SVD tends to concentrate recommendations on a smaller subset of items that are well represented in the latent space, which reduces its coverage.

Model	P@10	HR@10	MAP@10
UserCF KNN	0.0028	0.0280	0.0020
ItemCF KNN	0.0032	0.0309	0.0019
SVD	0.0058	0.0554	0.0032
TopPop	0.0291	0.2491	0.0223

Table 10: Top-10 recommendation metrics

Model	MRR@10	Coverage
UserCF KNN	0.0095	0.5268
ItemCF KNN	0.0098	0.4892
SVD	0.0153	0.1437
TopPop	0.0889	0.0203

Table 11: Top-10 recommendation metrics

The next step was to select the top 20% of users who are the most active in the training dataset, as well as the 20% of users who are the least active. We then recalculated the Hit Rate@10 for all the models and summarized the results in Table 12.

Model	Top 20%	Bottom 20%
UserCF KNN	0.0397	0.0144
ItemCF KNN	0.0614	0.0181
SVD	0.1083	0.0361
TopPop	0.4116	0.2022

Table 12: Hit Rate@10 for top and bottom 20% of items

The Hit Rate@10 for the top 20% most active users shows that SVD outperforms UserCF and ItemCF, indicating that matrix factorization can better capture latent preferences. However, TopPop achieves the highest hit rate for these users due to recommending the most popular items, which are more likely to appear in the test set. For the bottom 20% least active users, all collaborative filtering models show very low Hit Rate, which demonstrates the cold-start problem. In contrast, TopPop remains relatively stable by being the model performing the best, demonstrating that popularity-based recommendations are more robust for users

with little interaction history. Overall, this shows that collaborative filtering models perform much better with active users but struggle with sparse user profiles, whereas popularity-based approaches are more consistent however not personalized.

We performed a similar analysis, this time focusing on items instead of users. We compared the top 20% most interacted-with items with the bottom 20% least interacted-with items and computed the coverage for each group. The results are shown in Table 13.

The coverage results show clear differences between the models. For the top 20% most popular items, UserCF has the highest coverage, followed by ItemCF and SVD, while TopPop covers the fewest items. For the bottom 20% least popular items, UserCF and ItemCF have much higher coverage, whereas SVD and TopPop cover very few items. The high coverage of less popular items for UserCF and ItemCF might be due to the fact that these models have more chances to recommend niche items based on direct similarities between users or items. Even items with few ratings can appear in a user's top-10 recommendations if they are similar to the user's preferences, which increases their coverage. On the other hand, SVD prefers items with enough ratings to learn latent factors, so it rarely recommends very niche items. TopPop always recommends the most popular items, so it completely ignores the long-tail. Overall, this shows that UserCF and ItemCF give more diverse recommendations in terms of item variety, while SVD and TopPop are more biased towards popular items.

Model	**Top 20%**	**Bottom 20%**
UserCF KNN	0.2795	0.6827
ItemCF KNN	0.2043	0.6827
SVD	0.2043	0.0752
TopPop	0.1021	0.0000

Table 13: Coverage for the top and bottom 20% of items

Regarding the error analysis for neighborhood-based UserCF, we selected two reference users from the training set based on their Mean Reciprocal Rank (MRR) on the test set : one with a high MRR (well-predicted) and one with a low MRR (poorly-predicted). For the high MRR user, we observed that several neighbors shared multiple items with the reference user (up to 4 items), which allowed the model to accurately predict most ratings (e.g., true ratings of 5.0 predicted as 5.0 or 4.66). This indicates that neighborhood-based CF works well when the reference user has neighbors with overlapping preferences. In contrast, the low MRR user had very limited overlap with neighbors (only 1 item in common per neighbor), resulting in poorer predictions (e.g., true ratings of 5.0 predicted as 3.39, and 1.0 predicted as 2.42). This demonstrates that the model struggles for users with unique or sparse rating histories, where neighbors provide little useful information. Overall, this analysis highlights that neighborhood-based CF is effective for users whose preferences are represented by similar neighbors, but it performs poorly for long-tail or atypical users, which explains the difference in MRR scores observed between the two users.

4 Week 9 : Text Representation

For this part, we decided to use only the items present in the train and test splits. This choice is mainly motivated by computational considerations, as the full metadata file is quite large. Filtering allows us to save memory and speed up processing. We also decided to limit the amount of characters in each description to 500 characters, also to gain memory storage during the process. Although using the entire dataset could provide more textual context and potentially improve the quality of NLP representations, we consider this subset sufficient for learning meaningful item embeddings while keeping resource usage reasonable.

For preprocessing the textual descriptions of each video games, we applied several standard steps to clean the data :

- **Lowercasing :** All text was converted to lowercase to ensure that words like "Game" and "game" are treated as the same
- **Tokenization :** The text was split into individual words (tokens) to allow further processing at the word level
- **Stopword removal :** Common words such as "and", "the", or "of" were removed since they do not carry significant semantic meaning and add noise
- **Punctuation removal :** Punctuation marks were removed to avoid irrelevant tokens that do not contribute to the model

- **Stemming :** Words were reduced to their root forms to combine variations of the same word and reduce vocabulary size

These preprocessing steps help significantly reduce the total number of unique words while retaining the most meaningful information for NLP models, improving both efficiency and generalization. Table 14 displays the difference in the vocabulary size between before and after words preprocessing.

Vocabulary	Number of words
Before preprocessing	25,078
After preprocessing	15,505

Table 14: Vocabulary size before and after preprocessing

After preprocessing the textual data, we transformed the item descriptions into numerical vector representations using two different approaches. First, we applied the TF-IDF method, which represents each item as a vector based on the frequency and importance of words within the corpus. In parallel, we trained a Word2Vec model on the preprocessed corpus to learn dense vector representations of words based on their contextual usage. Each item was then represented by averaging the embeddings of the words in its description.

To explore item similarity, we computed cosine similarity between item representations obtained using both TF-IDF and Word2Vec embeddings. We then compared the most similar items for a given reference item, which is the index=0. Table 15 presents the set of predicted most similar items to the reference item.

Word2Vec Similar Items	TF-IDF Similar Items
76	461
564	17
873	4
106	909
638	567
103	114
904	839
2	842
3	3

Table 15: Most similar items to the reference item using Word2Vec and TF-IDF

We observe that the two methods return different sets of similar items, with only a small overlap which is the item 3 appearing in both results. This highlights the fundamental difference between the two representations. TF-IDF relies on exact word matching and term frequency. As a result, it tends to retrieve items that share many identical words with the reference item. This makes it highly sensitive to the specific vocabulary used in the descriptions. In contrast, Word2Vec captures semantic relationships between words by learning from their context. Therefore, it is able to identify items that are similar in meaning, even if they do not share the exact same words. This often leads to more diverse and semantically relevant recommendations.

5 Week 10 : Content-Based Recommender System

For this part of the project, we extended the item representation by including additional textual features alongside the description. Specifically, we incorporated the title and the brand of each item. We selected these attributes because they have a relatively limited range of possible values, which makes them easier to model, and yet they provide meaningful insights about the nature of the product. The title often contains key terms that summarize the item, while the brand can convey information about style, quality, or category, helping to improve item differentiation and the quality of content-based recommendations. We applied standard text preprocessing steps to all features, including lowercasing, tokenization, stopword removal, punctuation removal, and stemming. As we saw earlier, these steps help reduce noise, eliminate irrelevant words, and normalize terms, which improves the quality of the textual representation. Finally, the processed text was transformed into a TF-IDF vector representation, allowing each item to be represented numerically based on the importance of its terms within the dataset. To represent users in the same vector space as items, we computed a vector representation for each user by aggregating the vectors of the items they have rated in the training set. As a result, the user vector captures the user's preferences in terms of item content.

To predict user-item interactions, we computed the similarity between user and item representations using cosine similarity. This metric measures the angle between two vectors and is particularly well-suited for high-dimensional sparse data such as TF-IDF representations. Unlike distance-based measures such as Euclidean distance, cosine similarity focuses on the orientation of vectors rather

than their magnitude, making it more robust for text-based representations. The resulting similarity score is used as a proxy for the user's preference for a given item.

We evaluated the performance of the content-based recommender system by predicting each user's item preferences based on cosine similarity between user and item vectors. The recommendations were then assessed using several ranking metrics, including Precision, Mean Reciprocal Rank, Mean Average Precision, Hit Rate, and Coverage. Only items with a rating greater than or equal to 3 in the test set were considered relevant. These results are shown in tables 16 and 17.

Model	Precision@10	MRR@10
Content-Based	0.0063	0.0142

Table 16: Performance of the content-based recommender system

Model	MAP@10	HR@10	Coverage
Content-Based	0.0031	0.0598	0.0064

Table 17: Performance of the content-based recommender system

Comparing the content-based recommender system with the collaborative filtering models from previous weeks, we observe that it generally achieves slightly higher performance on ranking metrics such as Precision@10, MAP@10, and MRR@10 compared to UserCF, ItemCF, and SVD. This improvement can be attributed to the use of item content features, which allows the model to better capture the characteristics of each item and match them to user preferences even in cases where there is limited rating data. However, the coverage remains very low, indicating that only a small fraction of items in the catalog are actually recommended across all users. This might be due to the fact that the model focuses on items closely related to each user's profile and does not naturally explore the full item catalog. In contrast, popularity-based methods like TopPop, although biased towards popular items, achieve higher hit rates but very low coverage. Overall, the content-based system exhibits the expected trade-off between accuracy and diversity: it slightly improves personalization and ranking metrics compared to standard collaborative filtering models, but recommends only a very small subset of items, reflecting limited exploration of the full catalog.

6 Week 11: Hybrid Recommender Systems

For this part, we combined the user-based collaborative filtering (User CF) model with a content-Based (CB) model leveraging item features (title, brand, and description). The User CF model predicts a user's preference for an item based on the ratings of similar users, while the Content-Based model estimates preferences by computing the similarity between user preference vectors and item feature vectors. Three hybrid strategies were implemented :

1. **Parallel (Weighted Sum)**: This method computes predictions independently for both models and then combines the scores for each item using a weighted sum, giving slightly more weight to the content-based model, which previously showed higher individual performance. The top-ranked items after combination form the final recommendation list.

2. **Switching (Round-Robin)**: In this approach, recommendations alternate between the two models for each user in a round-robin manner. The method ensures that items from both User CF and CB appear in the final top-k list, enhancing diversity while still leveraging the strengths of both models.

3. **Pipelining (Sequential)**: This strategy uses the output of the User CF model as a filter : only items recommended by CF are scored by the CB model. The final ranking is based on the CB scores of these pre-selected items, effectively refining the CF recommendations with content-based information.

The results of each one of these strategies are shown in table 18 amd 19.

Model	Precision@10	MRR@10
Parallel Weighted Sum	0.0143	0.0456
Round-Robin	0.0045	0.0102
Pipelining	0.0207	0.0665

Table 18: Precision and MRR for hybrid recommender systems

The results highlight clear differences between the three hybrid strategies. The pipelining approach

Model	MAP@10	Hit Rate@10	Coverage
Parallel Weighted Sum	0.0116	0.1281	0.0041
Round-Robin	0.0022	0.0439	0.0060
Pipelining	0.0171	0.1814	0.0032

Table 19: MAP, Hit Rate and Coverage for hybrid recommender systems

achieves the best performance across all ranking metrics, with the highest Precision@10, MRR@10, MAP@10, and Hit Rate@10. This confirms that using collaborative filtering as a candidate generator and refining the ranking with content-based similarity is highly effective. The CF model selects relevant items based on user behavior, while the CB model improves the ranking by leveraging item features, leading to more accurate recommendations. The parallel with weighted sum model also performs well, achieving intermediate results. In contrast, round-robin switching strategy yields the lowest performance on all ranking metrics. The method does not consider the relevance scores when merging recommendations, but simply alternates between models. While this increases diversity, it introduces less relevant items into the top-k list, reducing overall accuracy. Regarding the coverage, an opposite trend is observed. The round-robin method achieves the highest coverage, as it forces the inclusion of items from both models, promoting diversity. On the other hand, the pipelining approach has the lowest coverage, since it restricts recommendations to items pre-selected by the CF model, leading to a more concentrated set of items.